2026-5-6 C++内存管理

- 1.C/C++内存分布
- 2.C语言中动态内存管理的方式
  - 2.1malloc
  - 2.2 calloc
  - 2.3 realloc
  - 2.4 free
- 3.C++内存管理的方式
  - 3.1 new/delete操作内置类型
  - 3.2 new/delete操作自定义类型
- 4.operator new与operator delete函数
  - 4.1operator new与operator delete函数
- 5.new和delete的实现原理
  - 5.1 内置类型
  - 5.2自定义类型
    - 5.2.1 new的原理
    - 5.2.2 delete的原理
    - 5.2.3 new T[N]的原理
    - 5.2.1 delete[ ]的原理
- 6.定位new表达式（placement-new）
- 7.malloc/free和new/delete的区别

1.C/C++内存分布

内存分布说明：

1. 栈又叫堆栈-非静态局部变量/函数参数/返回值等，栈是向下增长的
2. 内存映射段是高效的I/O映射方式，用于装载一个共享的动态内存库。用户可以使用系统接口创建共享内存，叫做进程将通信。
3. 堆用于程序运行时动态内存分配，堆是可以向上增长的。
4. 数据段--存储全局数据和静态数据
5. 代码段--可执行的代码/只读常量

2.C语言中动态内存管理的方式

2.1malloc

这个函数向内存申请一块连续可用的空间，并返回指向这块空间的指针。若开辟失败则返回一个NULL所以malloc使用的时候一定要检查。
返回值的类型是void*,所以malloc函数并不知道开辟的空间的类型，所以需要强制类型转化。

```
void* malloc(size_t size);
```

2.2 calloc

开空间，并将空间全部初始化为0

```
void* calloc (size_t num,size_t size);
```

num：是多少个该空间
size：是一个空间开多大

2.3 realloc

如果你觉得你原本开辟的空间小了，可以用它增加你原本空间的大小，并保留好你原本空间的值，所以这个分为两种：

1. 原地扩容：你原本开辟的这块内存后还有足够的空间能成为你需要的新空间的大小
2. 异地扩容：如果原本地址的空间扩容后空间不够，则保留数据重新寻找孔家
   最直观的表现是，原地扩容返回的地址和原本的地址相同，异地扩容返回的地址和原本的地址是不一样的，因为重新寻找了空间.

```
void* realloc(void* ptr,size_t size);
```

ptr：是要调整的空间地址
size：是要调整为的空间的大小

2.4 free

动态内存的释放与回收

```
void free(void* ptr)
```

小测试：

```
void Text1() {
    int* p2 = (int*)calloc(4, sizeof(int));
    int* p3 = (int*)realloc(p2, sizeof(int)*10);
    free(p3);
}
```

这里的p2需要释放吗?
不需要，这里p3对p2进行了异地扩容，异地扩容的时候p2已经被释放了。

3.C++内存管理的方式

c语言中内存管理的方式在C++中可以继续使用，但是在 无能为力，因此C++又提出了自己的内存管理方式：**通过new和delete操作符进行动态内存管理**。

3.1 new/delete操作内置类型

```
void Text2() {
    //动态申请一个int类型的空间
    int* ptr1 = new int;
    //动态申请了一个int类型的空间并初始化为10
    int* ptr2 = new int(10);
    //动态申请是个int类型的空间
    int* ptr3 = new int[10];
    delete ptr1;
    delete ptr2;
    delete[] ptr3;
}
```

注意new和delete要匹配起来使用。

3.2 new/delete操作自定义类型

在申请自定义类型的空间时，new会调用构造函数，delete会调用析构函数，而malloc和free不会。

```
class A
{
```

```
public:
    A(int a = 0)
        :_a(a)
    {
        cout << "A():" << this << endl;
    }
    ~A()
    {
        cout << "~A():" << this << endl;
    }
private:
    int _a;
};
void Text03() {
    //new/delete和malloc/free最大的区别是new/delete对于自定义类型，除了开空间还会调用
    //构造函数和析构函数
    A* p1 = (A*)malloc(sizeof(int));
    A* p2 = new A(1);
    free(p1);
    delete p2;

    //内置类型几乎是一样的
    int* p3 = (int*)malloc(sizeof(int));
    int* p4 = new int;
    free(p3);
    delete p4;

    A* p5 = (A*)malloc(sizeof(A)* 10);
    A* p6 = new A[10];
    free(p5);
    delete[] p6;
}
```

4.operator new与operator delete函数

4.1operator new与operator delete函数

new和delete是用户进行动态内存申请和释放的操作符，operator new和operator delete是系统提供的全局函数，new在底层调用operator new全局函数来申请空间，delete在底层通过operator delete全局函数来释放空间。

1. operator new实际也是通过malloc来申请空间，如果malloc申请空间成功就直接返回，否则执行用户提供的空间不足以应对措施，如果用户提供该措施就继续申请否则就抛异常。

2. operator delete最终是通过free来释放空间的。

实际上operator new也是通过malloc来申请空间，如果malloc申请空间成功就直接返回，否则执行用户提供的空间不足的应对措施，如果用户提供就继续执行，否则抛异常。operator delete最终是通过free来释放空间的。

5.new和delete的实现原理

5.1 内置类型

如果申请的是内置类型的空间，new和malloc相似，delete和free相似，不同的是：new/delete申请和释放的是单个元素的空间，new[]和delete[]申请的是连续的空间，而且new在申请空间失败时会抛异常，malloc会返回NULL。

5.2自定义类型

5.2.1 new的原理

1. 调用operator new函数申请空间
2. 在申请的空间上执行构造函数，完成对象的构造

5.2.2 delete的原理

1. 在空间上执行析构函数，完成对象中资源的清理工作
2. 调用operator delete函数释放对象空间

5.2.3 new T[N]的原理

1. 调用operator new[ ]函数,在operator new[ ]中实际调用operator new函数完成N个对象空间申请
2. 在申请的空间上执行N次构造函数

5.2.1 delete[ ]的原理

1. 在释放的对象空间上执行N次析构函数，完成N个对象中资源的清理
2. 调用operator delete[]释放空间，实际在operator delete[]中调用operator delete来释放空间。

6.定位new表达式（placement-new）

定位new表达式是在已分配的原始内存中调用构造函数初始化一个对象。

1. 使用格式：
   new (place_address) type或者newnew (place_addres）type(initializer-list)
   place_address必须是一个指针，initializer-list是类型的初始化列表
2. 使用场景：

```
void Text04() {
    //定位new/replacement new
    //p1现在执行的只不过是A对象 相同大小的一段空间，还不能算是一个对象，因为构造函数没有执行
    A* p1 = (A*)malloc(sizeof(A));
    new(p1)A;
    p1->~A();
    free(p1);
    A* p2 = (A*)operator new(sizeof(A));
    new(p2)A(10);
    p2->~A();
    operator delete(p2);
}
```

定位new表达式在实际中一般是配合内存池使用的。因为内存池分配出的内存没有初始化，所以如果是自定义类型的对象，需要使用new的定义表达式进行显示构造函数进行初始化。

7.malloc/free和new/delete的区别

malloc/free和new/delete的共同点是：都是从堆上的空间申请，并且需要用户手动释放。不同的地方是：

1. malloc和free是函数，new和delete是操作符
2. malloc申请的空间不会初始化，new可以初始化
3. malloc申请空间时，需要手动计算空间大小并传递，new只需要在其后面跟上空间的类型即可如果是多个对象，[]中指定个数即可
4. mallocde的返回值为void*，在使用时必须强转，new不需要，因为new后面跟的是空间的类型
5. malloc申请空间失败时，返回的是NULL，因此使用时必须判空，new不需要，但是new需要捕获异常

6. 申请自定义类型对象时，malloc/free只会开辟空间，不会调用构造函数与析构函数，而new在申请空间后会调用构造函数完成对象的初始化，delete在释放空间前会调用析构函数完成空间中资源的释放清理。